

Anweisung: Übergang von einem Programmzustand zum nächsten.

Programmzustand: Daten im Speicher + Befehlszähler

Kontrollfluss: Reihenfolge, in der die Operationen des Programms abgearbeitet werden.

Datenfluss: Übergabe der Daten von einer Operation an eine andere Operationen

Zuweisung

$x = 5;$ $\leftarrow$ Voraussetzung: Variable und Ausdruck müssen "kompatible" Typen haben

$x = x + 1;$ $\leftarrow$ Wert von x wird um 1 erhöht

$x += 10;$ $\leftarrow$ Kurzschreibweise für $x = x + 10;$

Genauer für: $x = (\text{int}) x + 10;$
wenn x den Typ `int` hat

$x = (\text{byte}) x + 10;$
wenn x den Typ `byte` hat

etc.

d.h. `byte a;`



a ~~=~~ a+1; Typfehler
 byte : int

a += 1; ok, denn das ist Kurzschreibweise für
 $a = (\text{byte})(a + 1)$

x *= 2; ← Kurzschreibweise für $x = x * 2$;

$\begin{matrix} x++ \\ ++x \end{matrix} \}$ Kurzschreibweise für $x += 1$;

$\begin{matrix} x-- \\ --x \end{matrix} \}$ analog

Man kann manche Anweisungen auch als Ausdruck verwenden:

$x++$ hat als Wert den "alten" Wert von x

$++x$ hat als Wert den "neuen" Wert von x

Bsp: $\begin{matrix} x = 4; \\ y = x++; \end{matrix} \}$ y ist 4
 x ist 5

$\begin{matrix} x = 4; \\ y = ++x; \end{matrix} \}$ y ist 5
 x ist 5

Guter Stil: Anweisungen wie $x++$ oder $++x$

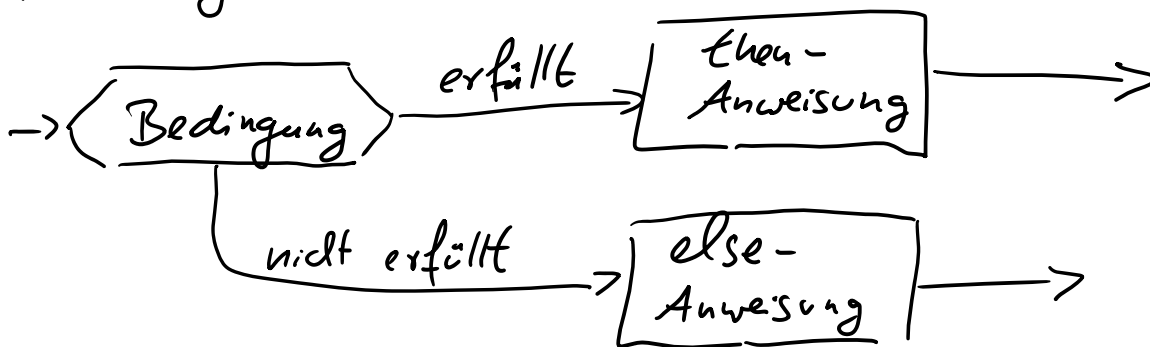
Guter Stil: Anweisungen wie $x++$ oder $++x$ sollten nicht als Ausdruck verwendet werden.

Ausnahme: $x = y = 5;$

steht für $x = (y = 5);$

if-Anweisung

Flussdiagramm



else-Anweisung darf fehlen

Bei geschachtelten if-Anweisungen:

- "else" gehört zum innersten "if"
- $\{ \dots \}$ Block von Anweisungen und Variablendeklarationen

Wenn "else" zum äußeren "if" gehören soll, dann muss man $\{ \dots \}$ einsetzen.

Alle Anweisungen enden mit $;$ oder mit $\}$.

Switch-Anweisung

für größere Fallunterscheidungen, die vom
1 2 ... 10 ... 100 abhängen.

für größere Fallunterscheidungen, die vom Wert eines Ausdrucks abhängen

```
switch ( byte-, short-, int-,  
         char- oder String-Ausdruck ) {  
    case Aus1, Aus'1 → Anw1  
    case Aus2      → Anw2  
    ...  
    case Ausn      → Anwn  
    default         → Anwn+1 }  
                                ↙ default-Fall  
                                kann fehlen
```

Man kann "switch" auch in Ausdrücken benutzen.

- Problem: Wie kann man einen "Ausdruck-Block" realisieren?
- Lösung: Erst Anweisungen + Variablen Deklarationen (wie in normalem Block), dann "Rückgabe" des gewünschten Ausdrucks mit "yield" am Ende

Beim switch-Ausdruck muss die Fallunterscheidung vollständig sein, ^{da} sonst sein Wert nicht definiert ist.

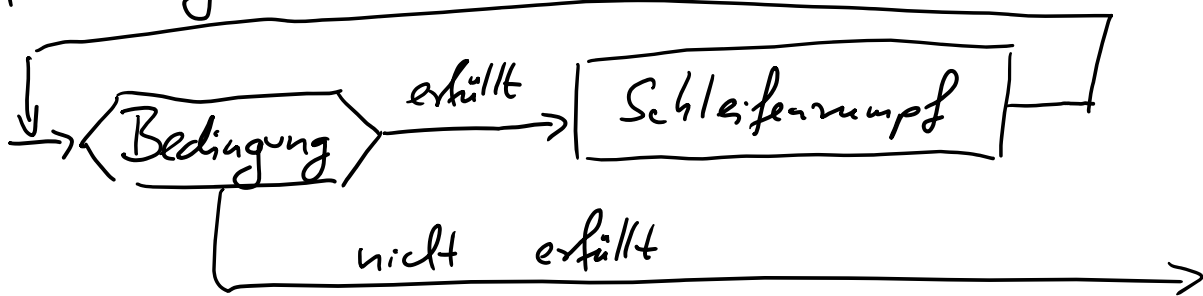
while-Schleife

→ 1 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20

while Schleife

Java hat 3 Arten von Schleifen: while, do, for
Sind gleichmächtig, aber manchmal ist eine lesbarer
als die andere.

Flussdiagramm:



Gefahr der Nicht-Terminierung, wenn die Bedingung
immer wieder erfüllt.

Bsp: Prim

Untersuche, ob n eine Primzahl ist (mit $n \geq 2$).
Dazu durchläuft teiler alle Zahlen von 2 bis

$$\lfloor \sqrt{n} \rfloor$$

größte int-Zahl,
die $\leq \sqrt{n}$ ist

z.B. $\lfloor \sqrt{5} \rfloor = 2$

"istPrimzahl" gibt an, ob sicher noch kein Teiler von n
gefunden wurde

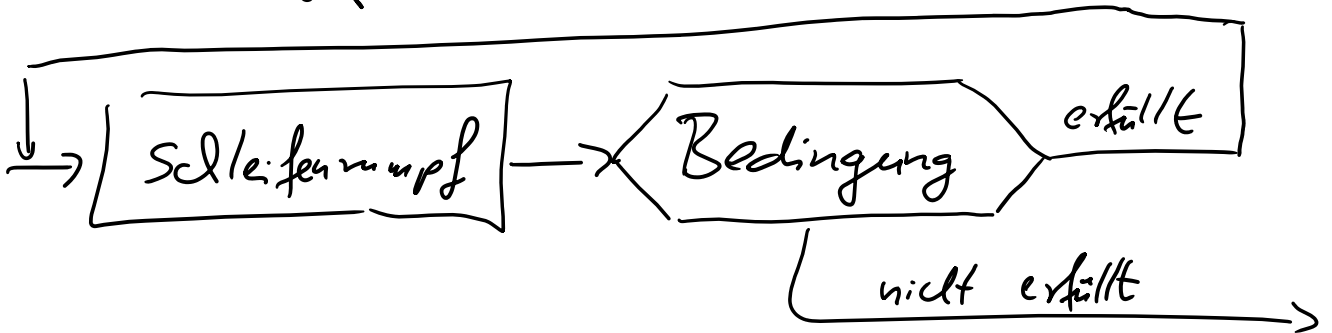
" $n \% \text{teiler}$ " ist die Modulo-Operation, d.h. der

" $n \% \text{ teiler}$ " ist die Modulo-Operation, d.h. der Rest sr der Division von n durch teiler

z.B. 11 % 3 ist 2
11 / 3 ist 3

do-Schleife

Schleife, bei der der Schleifenrumpf mindestens einmal ausgeführt wird.



Bsp: Wurzel

Ziel: Berechne $\sqrt{x}$ für $x \geq 1$

Ergebnis liegt im Intervall $[0, x]$.
 $\uparrow$ $\uparrow$
 u_G o_G

Führe Intervallschachtelung durch, bis das Intervall $\leq \varepsilon$ ist.

In jedem Schritt wird die Mitte des Intervalls berechnet:

$$m = \frac{16 + 06}{2}$$

1. Fall $m^2 > x$

$$\Rightarrow m > \sqrt{x}$$

$\Rightarrow$ Ersetze $[aG, oG]$ durch $[aG, m]$



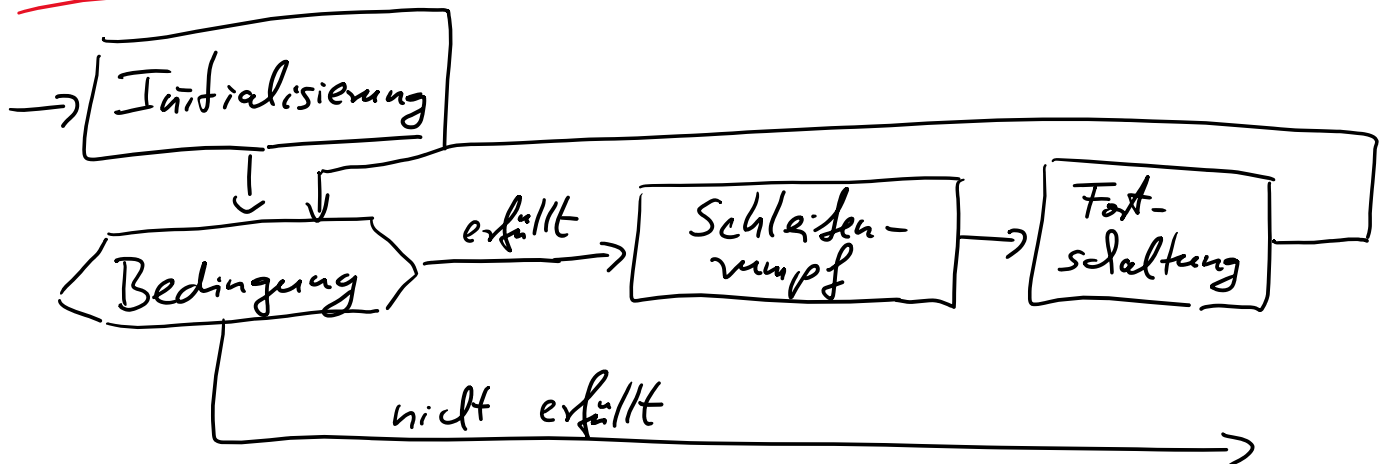
2. Fall $m^2 \leq x$

$$\Rightarrow m \leq \sqrt{x}$$

$\Rightarrow$ Ersetze $[aG, oG]$ durch $[m, oG]$



for-Schleife



Ist für Zählschleifen gedacht, bei denen ein Wert immer wieder erhöht oder erniedrigt wird, bis er einen bestimmten anderen Wert erreicht hat.

`for(int x=0; x <= 10; x++) {`

`⋮` $\leftarrow$ Hier steht x zur Verfügung

`}`

Initialisierung

Initialisierung

eine Variablen Deklaration
oder mehrere Zuweisungen
(mit Komma getrennt)

z.B. `int x=0, y=5`

oder `x=0, y=5`
(für bereits deklarierte
Variablen)

Fortschaltung

mehrere Zuweisungen
(mit Komma getrennt)

`x++, y--`

Bsp: Geschachtelte Schleife
(d.h. Schleifenumpf enthält auch eine Schleife)

i	j
1	1
2	1
2	2
3	1
3	2
3	3

Guter Stil: Nur Zählschleifen mit "for" programmieren.

Initialisierung + Fortschaltung sollten nur
das Weiterzählen der Zählvariablen betreffen.

Sprunganweisungen

weitere Art von Sprunganweisung:

weitere Art von Sprunganweisung:

`System.exit(n)` ← Sendet das Programm

wobei n eine Integer-Zahl ist.

Jedes $n \neq 0$ zeigt Fehler an.

break: beendet die aktuelle Schleife

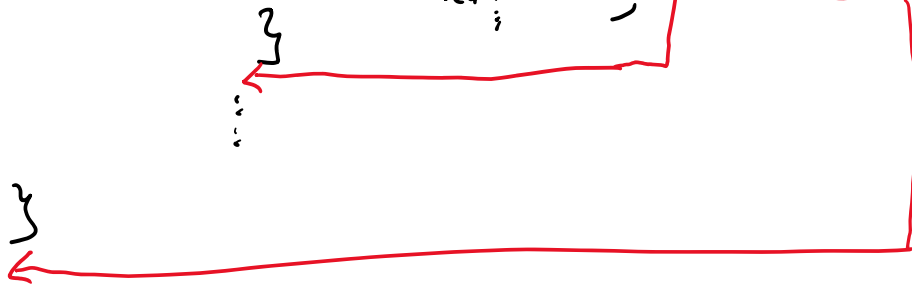
Bei geschachtelten Schleifen wird die innerste Schleife beendet.

Sonst: Verwendung von Labels

aussen: `while(...)` {

...
innen: `while(...)` {

...
... `break;`
... `break; aussen;`

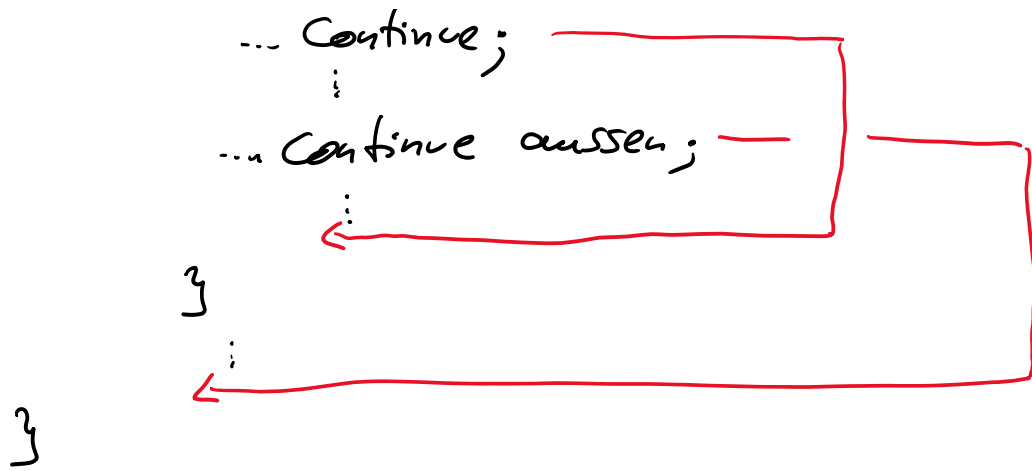


continue: springt ans Ende des aktuellen Schleifenrumpfs

aussen: `while(...)` {

...
innen: `while(...)` {

...
... `continue;`



Bsp: Berechne "Freitag, den 13."

MO	DI	MI	DO	FR	SA	SO
1	2	3	4	5	6	7

Der 31.12.2024 war DI (Gib 2 ein)

2025 ist kein Schaltjahr (Gib 0 ein)

Sprunganweisungen sollten zurückhaltend verwendet werden.

Manche Sprachen erlauben "goto":

Sprunganweisungen für beliebige Sprünge

Dijkstra: "goto considered harmful"

Führt oft zu unleserbaren Programmen.